Controller Development: Code generation, Requirements, and Continuous Integration

Sam Reinsel

22 April 2026

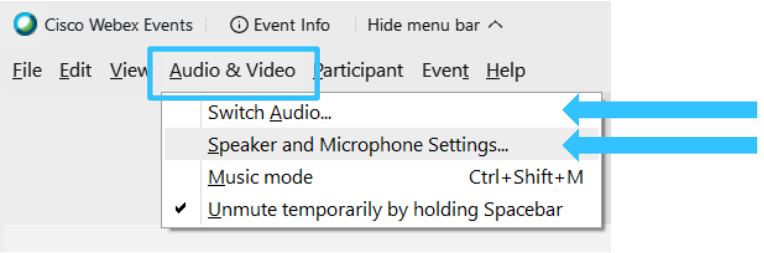
© 2026 The MathWorks, Inc.

The webinar will begin in half an hour

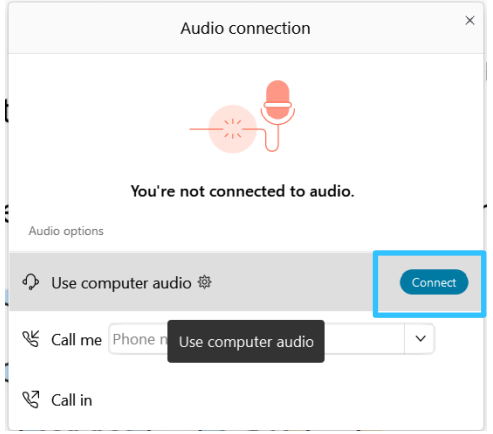
A recording of today's webinar will be emailed to you in about 5 business days

Joining the Audio

1. Click the **Audio & Video** tab at the top of the Webex menu bar

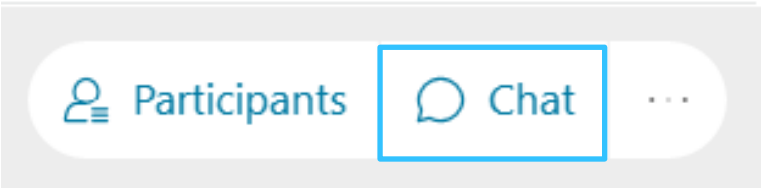


2. Select "**Speaker and Microphone Settings**" to make sure your speakers are not muted and the audio settings are correct.
3. Select "**Switch Audio**" if you are not connected to the Webex audio and click "**Connect**" next to "**Use computer audio**."



Technical Issues?

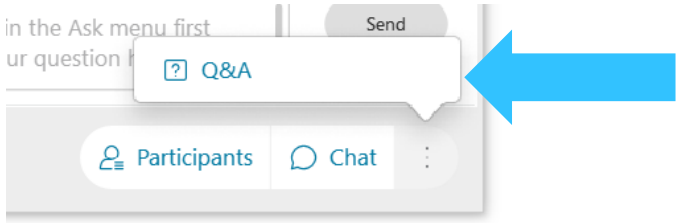
You can contact the host at any time using the **Chat Panel** in the lower right to troubleshoot technical issues.



Questions to the Presenter

You can pose your questions to the presenter at any time during the Webinar using the **Q&A Panel** on the right side of the Webex window.

Please note: the Presenter cannot see questions in the Chat Panel



Agenda

- Intro to Code generation in MATLAB and Simulink
- Embedded targets
- Primer on Simulink VnV tools
- Continuous integration testing

Code Generation from MATLAB and Simulink

Code Generation is Everywhere …

GM Standardizes on Model-Based Design for Hybrid Powertrain Development

"The Two-Mode Hybrid powertrain took Model-Based Design to a new level within GM. This project provided the confidence and experience we needed to apply MathWorks tools for Model-Based Design on other large-scale global engineering programs."

— *Kent Helfrich, General Motors*

Badge for GM's Two-Mode Hybrid powertrain, which is used in vehicles across several product lines.

Jaguar Reduces Development Costs with Rapid Prototyping and Code Generation

To reduce the costs of testing new product functionality, Jaguar uses MATLAB and Simulink products to generate control software and then deploys it on general-purpose ECUs on existing vehicles. As a result, Jaguar has increased its ability to test more design options and realized significant time and cost savings.

MPC555-based electronic control module, pictured on the vehicle dashboard. Photo courtesy of add2 Ltd.

BAE Systems Delivers DO-178B Level A Flight Software on Schedule with Model-Based Design

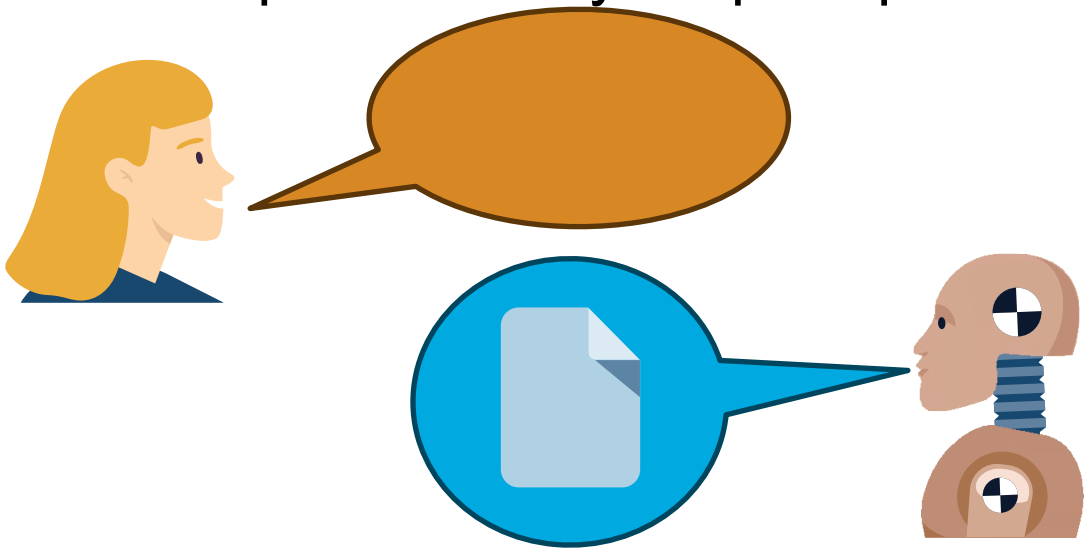
"When we generated code from our Simulink models with Embedded Coder, the team we handed it off to knew it was gold—that it was debugged and fully met the requirements—because we had run it through the Simulink test vectors supplied by our customer. That was a huge advantage on this program."

— *Maria Radecki, BAE Systems Electronic Solutions*

Primary flight control computers from BAE Systems.

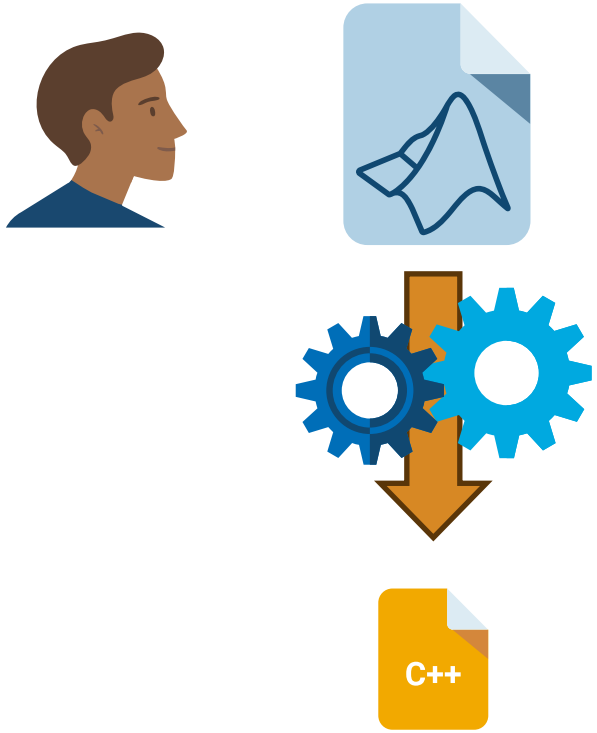
In the age of Generative AI, What do we mean by Code Generation?

- Generative AI *predicts* which outputs match your prompt



- The same prompt won't always result in the same code

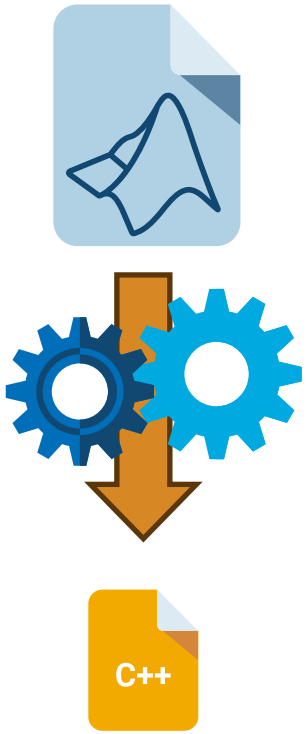
- Code Generation creates code deterministically based on your MATLAB code or Simulink models



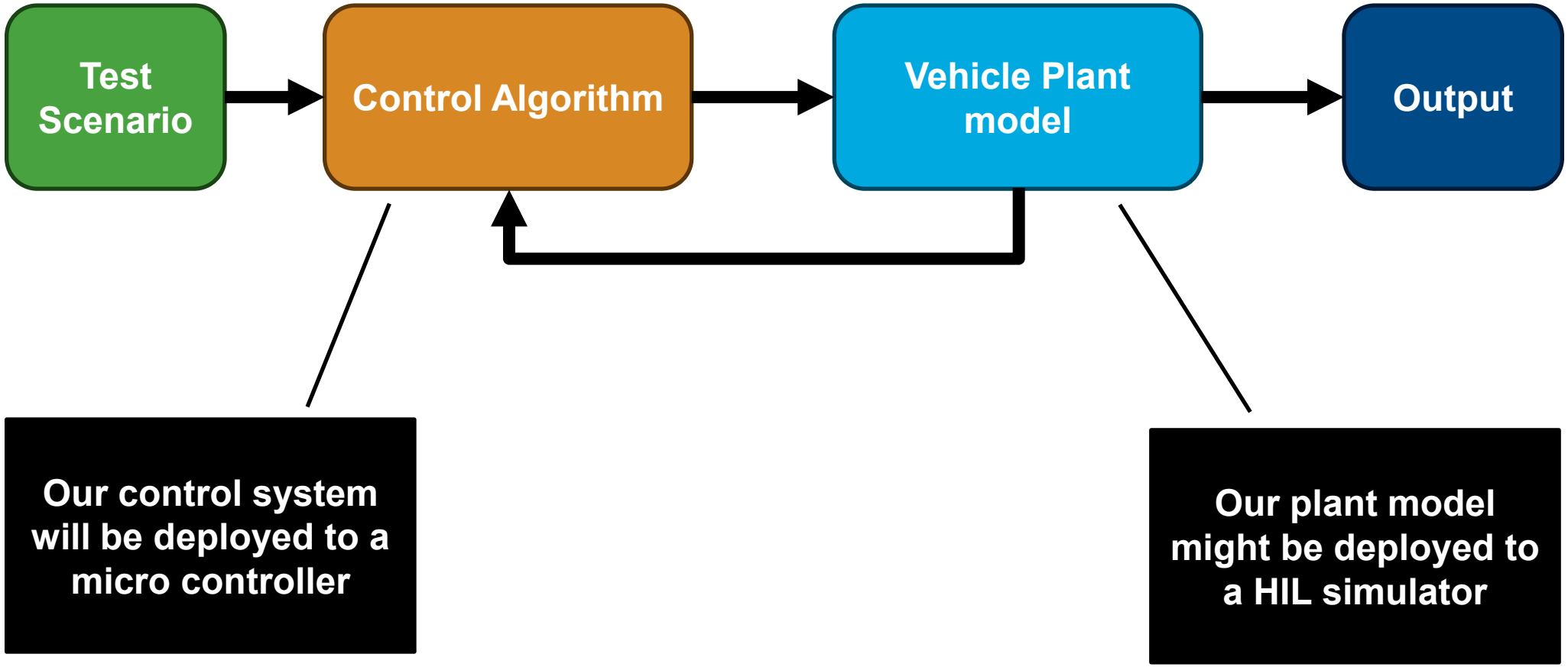
These days it doesn't have to be either/or!

MATLAB code generation is generally focused on

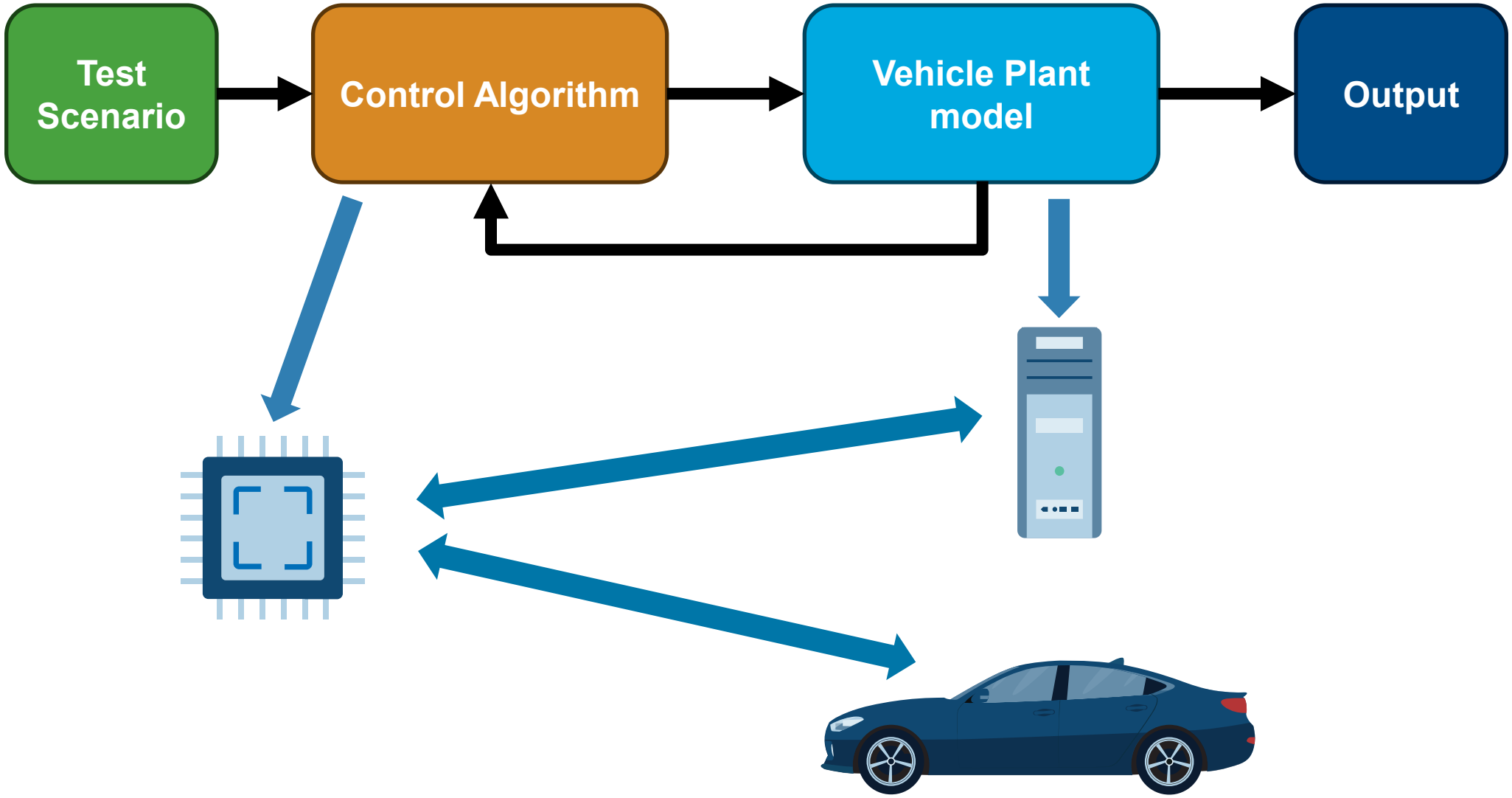
- Start algorithm development in MATLAB
- Rapidly iterate to solidify design and requirements
- Once ready to deploy, we can generate C/C++ code for deployment
  - Code reports can help trace MATLAB code to C code, if needed
- We can always tweak the MATLAB code and re-generate.



Simulink code generation is more focused on control systems we plan to deploy



Simulink code generation is more focused on control systems we plan to deploy



Want to look at the Files?

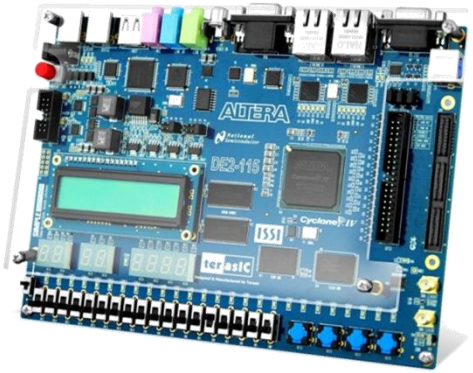
- https://github.com/mathworks/student-competition-code-generation-training
- https://www.mathworks.com/matlabcentral/fileexchange/69089-student-competition-code-generation-training

Direct Hardware support

Automatic Code Generation

ANSI **C/C++** for embedded systems

- MATLAB Coder
- Simulink Coder
- Embedded Coder



HDL for FPGAs

- HDL Coder
- HDL Verifier

Structured text for **PLC**s

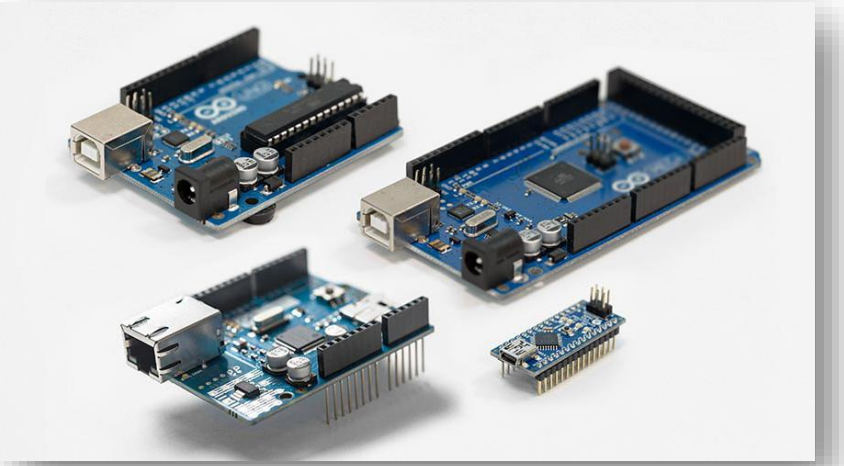
- Simulink PLC Coder

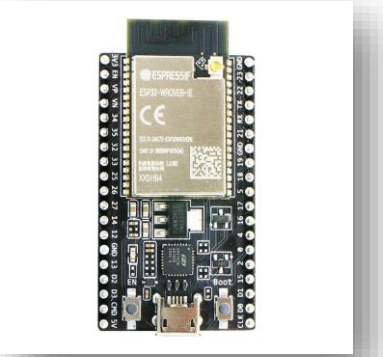
Programming **GPUs**

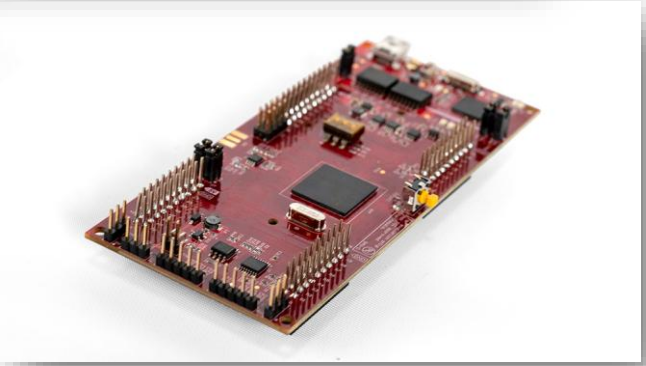
- GPU Coder

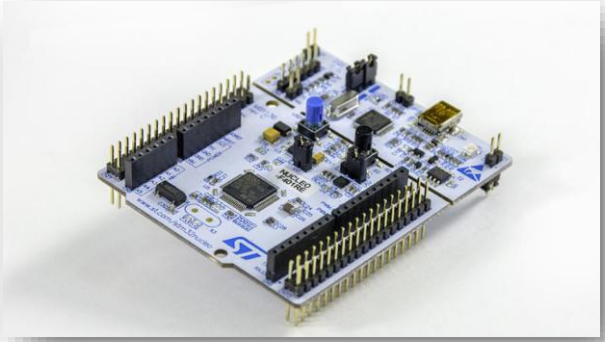
Common targets in Academia and student competitions

- Raspberry Pi boards
- Arduino (including Arduino compatible ESP32 and Pi boards)
- NI C2000
- STM boads (both nucleo and custom PCBs)









Contact Us How to Buy Carlos

MathWorks Products Solutions Academia Support Community Events

Hardware Support

Search Hardware Support | Hardware Support

Overview | Search Hardware Support | Request Hardware Support

Trial software | Contact sales

Refine by Product

MATLAB	13
Simulink	12
Audio System Toolbox	1
Communications System Toolbox	6
Computer Vision System Toolbox	1
Data Acquisition Toolbox	15
DSP System Toolbox	2

Refine by Product Family and Category

MATLAB Product Family	103
Simulink Product Family	121

Refine by Vendor

3D Robotics	1
3S-Smart Software Solutions	1
ADLINK	3
ARM	17
AUTOSAR Development Partnership	1
Adimec	3

Results 1 - 25 of 173

ADALM-PLUTO Radio Support from Communications System Toolbox
Prototype and test software-defined radio (SDR) systems using Analog Devices ADALM-PLUTO with MATLAB and Simulink
Vendors: Analog Devices
Tags: Support Package Installer Enabled, MathWorks Supported

Adimec Camera Support from Image Acquisition Toolbox
Use Adimec cameras with MATLAB and Simulink to acquire video and images.
Vendors: Adimec
Tags: MathWorks Supported

ADLINK Support from Data Acquisition Toolbox
Create your own waveforms, measurement and analysis routines, and applications for ADLINK DAQ hardware using MATLAB and Data Acquisition Toolbox
Vendors: ADLINK
Tags: Connections Program

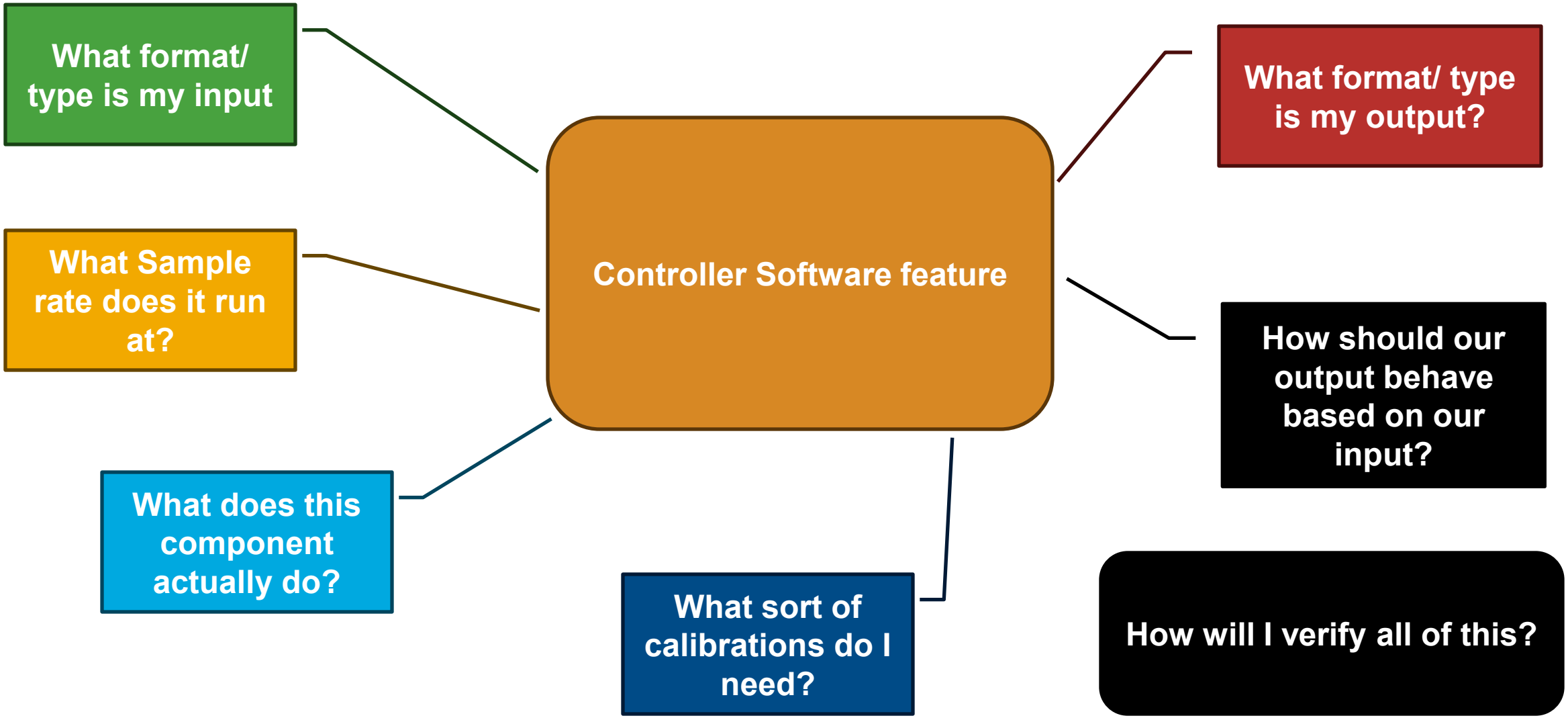
Allied Vision Camera Support from Image Acquisition Toolbox
Use AV cameras with MATLAB and Simulink to acquire video and images
Vendors: Allied Vision
Tags: MathWorks Supported

MathWorks

https://www.mathworks.com/hardware

Primer on Requirements and Simulink Test

Before we can add any interesting automation, we need to know some things about our software



Requirements

Writing Good Requirements

- A good requirement is **necessary**
  - Supports the objective of the system
  - Ask: “Whats the worst thing that happens if this requirement isn’t included?
- Example: “The battery module shall not exceed a mass of 100kg”
  - Is this necessary?
    - Probably: the mass of the module impacts assembly by your manufacturing team, and there are standards and limits for manual lifting of heavy objects

A good requirement should be **testable**

- Subjective statements like “easy, hard, difficult, quickly” are not verifiable

- Example:
  - “The trunk shall be easy to open”
    - Easy for who? When? What conditions? What measurement?
  - “The trunk closure shall require no more than 50Nm of force applied at the hatch handle to open the trunk when the vehicle is pointed uphill on a 10% grade”

Requirements should be implementation agnostic

- “The battery shall have a cell configuration of 15s2p”

- This requirement is implying a specific implementation!
- What defines the need for this configuration? Is it the energy content?
- How do we know that 15s2p is to few or two many for our system?

- What customer need is driving this?

A good approach is to follow a specific structure

Subject

- The feature, function, or system that will perform the action

Verb

- “Shall” is the standard
- Not “should, could, must, may”

Action & Object

- The action identifies what the system does to satisfy the requirement
- The object receives the action

Value

- Constrains the goal of the requirement (how long, how fast, what distance, etc)
- May be tweaked over project lifetime based on stakeholder needs

Conditions

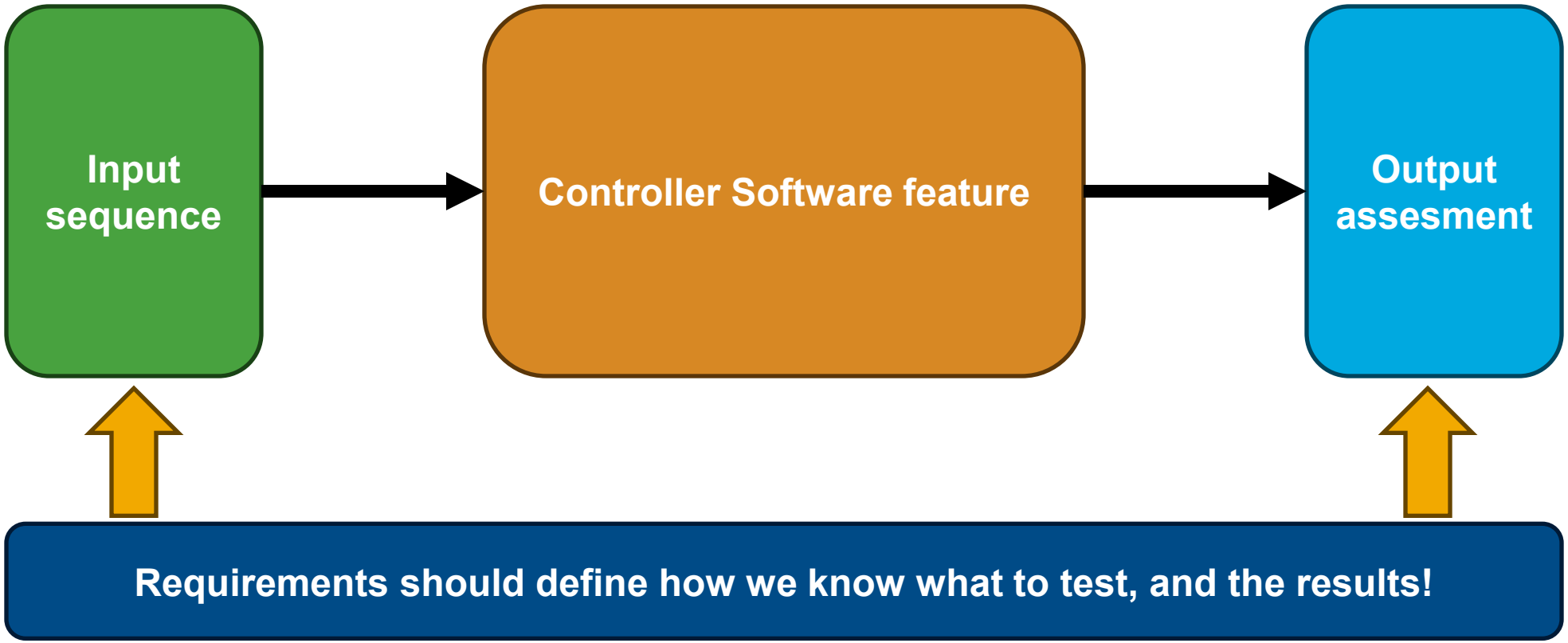
- Situations, circumstances in which the requirement must be met

Example:

The control system shall deactivate the battery within 1 second if the battery temperature exceeds 80 degrees C

Subject	The control system
Verb	shall
Action	deactivate
Object	the battery
Value	within 1 second
Conditions	if the battery temperature exceeds 80 degrees C

Testing our software is a bit more straightforward

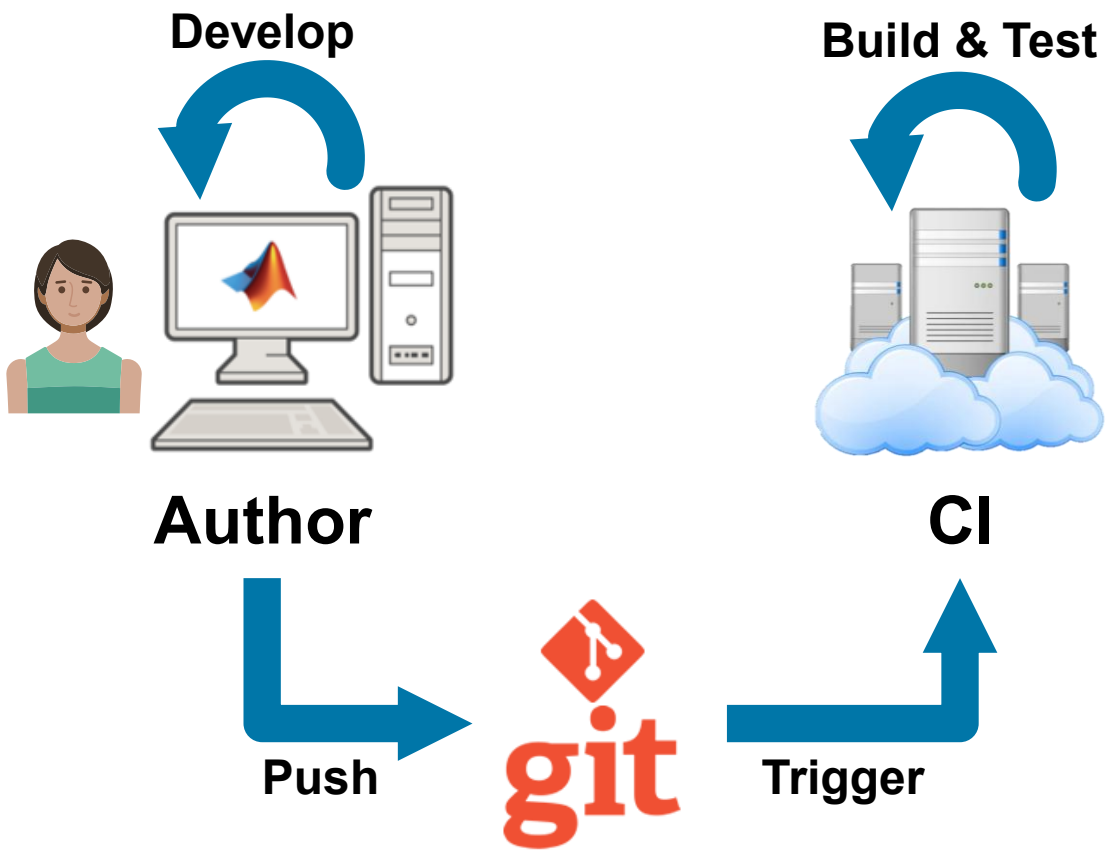


Continuous Integration

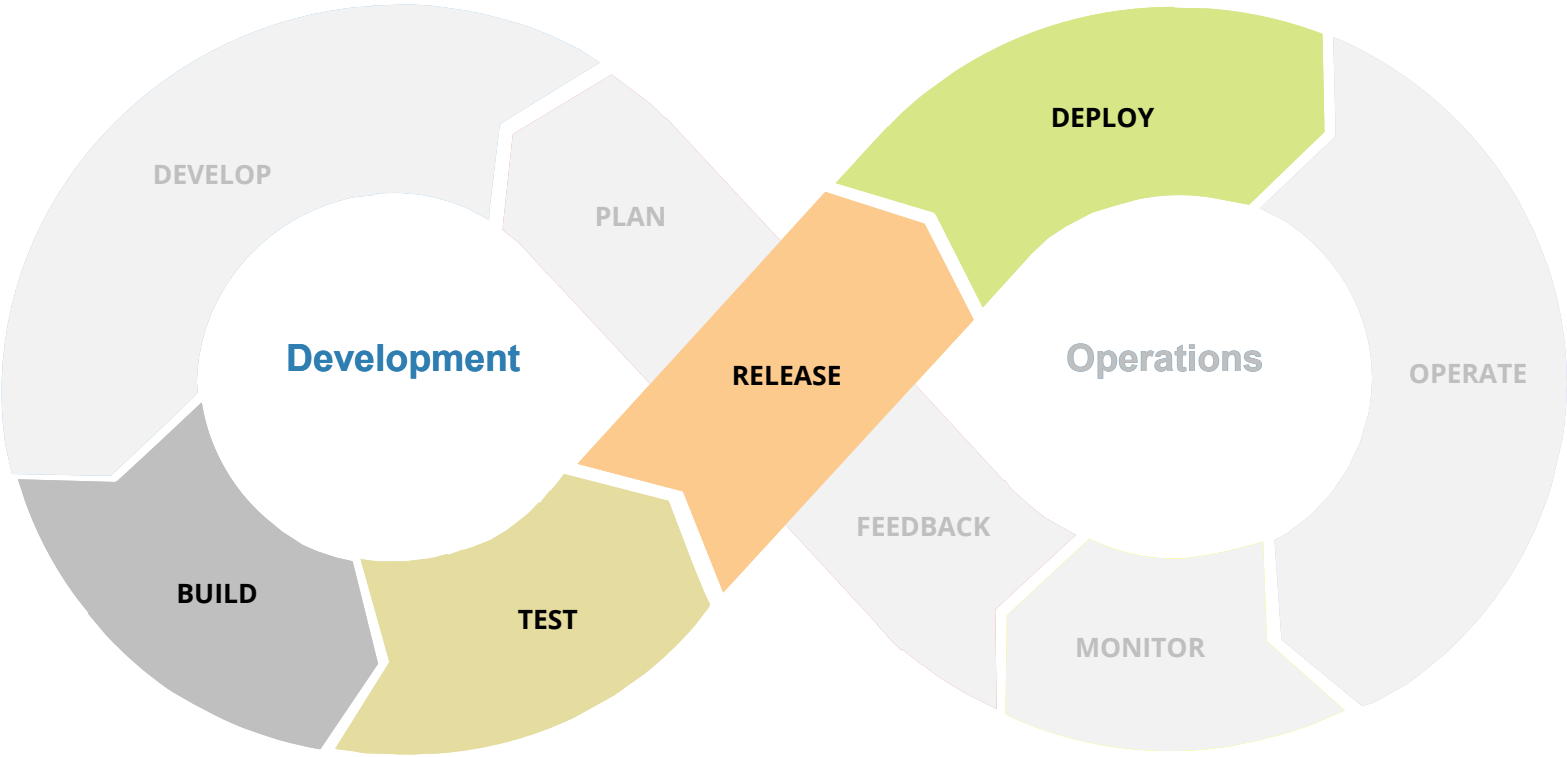
What is continuous integration (CI)?

- CI is a software practice that automates:
  - testing code changes
  - integrating code changes
  - deploying applications to production

- 3 major CI configurations
  - Fully in the cloud
  - Fully on-premises
  - Cloud-based orchestration with on-premises code execution



CI is part of a continuous software development process

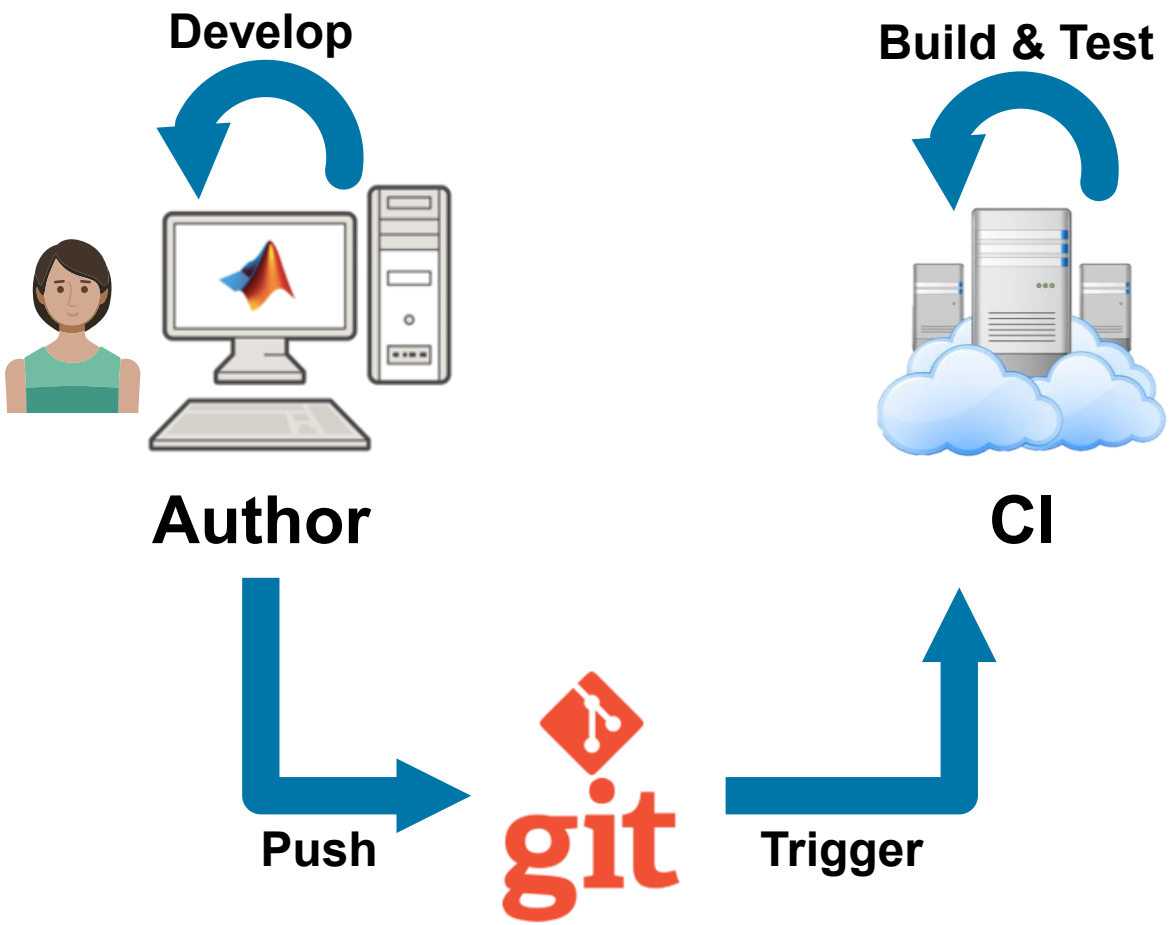


CI is focused on automating these steps

Why is everyone using CI?

- **Improved software quality**
  - Ensure every code change is tested automatically
  - Many ways to introduce quality gates to maintain high code quality
- **Reduced risk**
  - Catch bugs before they are integrated into your production codebase
  - Standardized build and release processes
- **Faster delivery** and **reduced costs**
  - Automate manual processes and offload compute
  - Streamline interactions between development, testing, IT, and operations

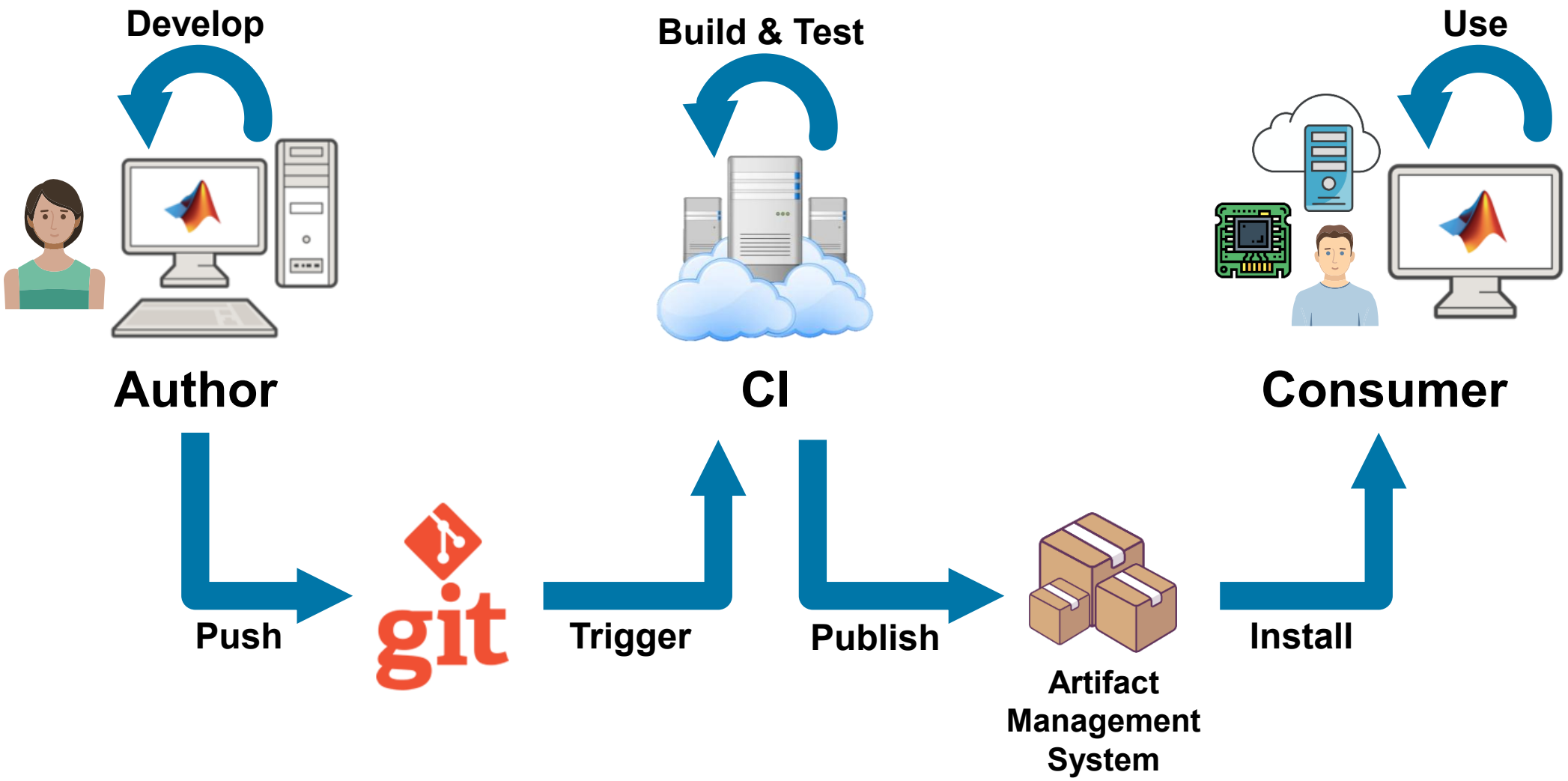
How does CI work?



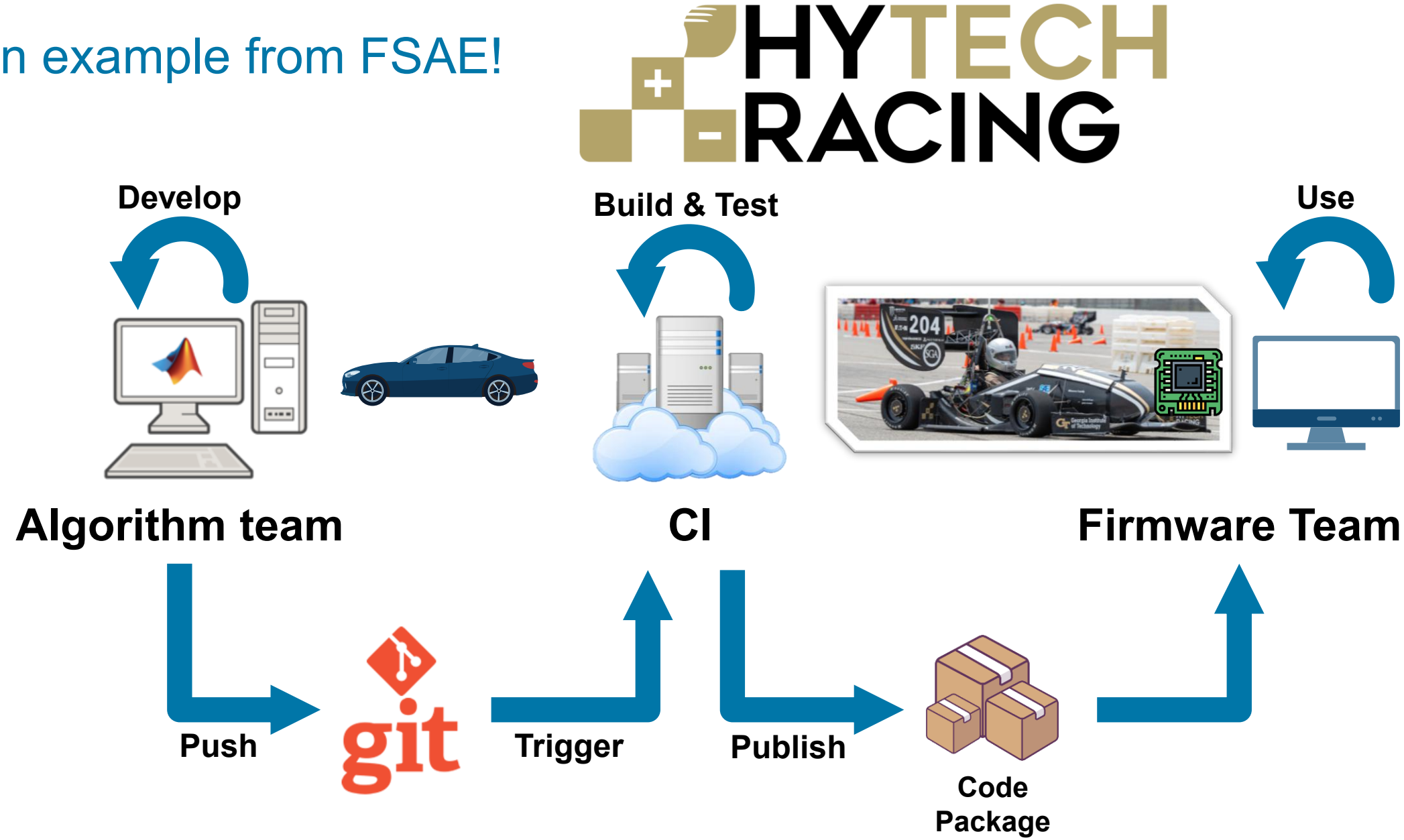
Wait… What are we building?

- MATLAB Toolbox
- Web App
- Generated C/C++
- Microservice
- EXE file
- Python package
- Java package
- .NET assembly
- …

How does CI work?



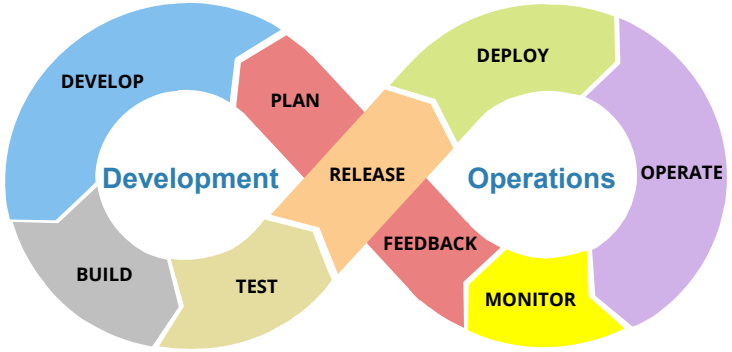
An example from FSAE!



3 Major Aspects to Consider

Getting MATLAB and Simulink into CI

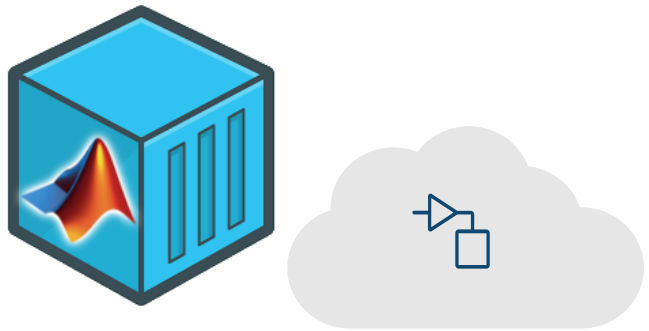
Licensing MATLAB and Simulink



Executing MATLAB and Simulink

Getting MATLAB and Simulink into your CI system

```
# Install MATLAB and Simulink
RUN wget -q https://www.mathworks.com/mpm/glnxa64/mpm && \
    chmod +x mpm && \
    ./mpm install \
        --release=R2025b\
        --destination=/opt/matlab \
        --products MATLAB Simulink && \
```



MATLAB Package Manager (`mpm`)

Scriptable tool to build container with selected components for memory efficiency and faster start-up

Containers & Reference Architectures

Dockerfiles, templates, and example scripts for roll-your-own integration

- Cloud Reference Architectures
- Dockerfile for creating a MATLAB container for CI

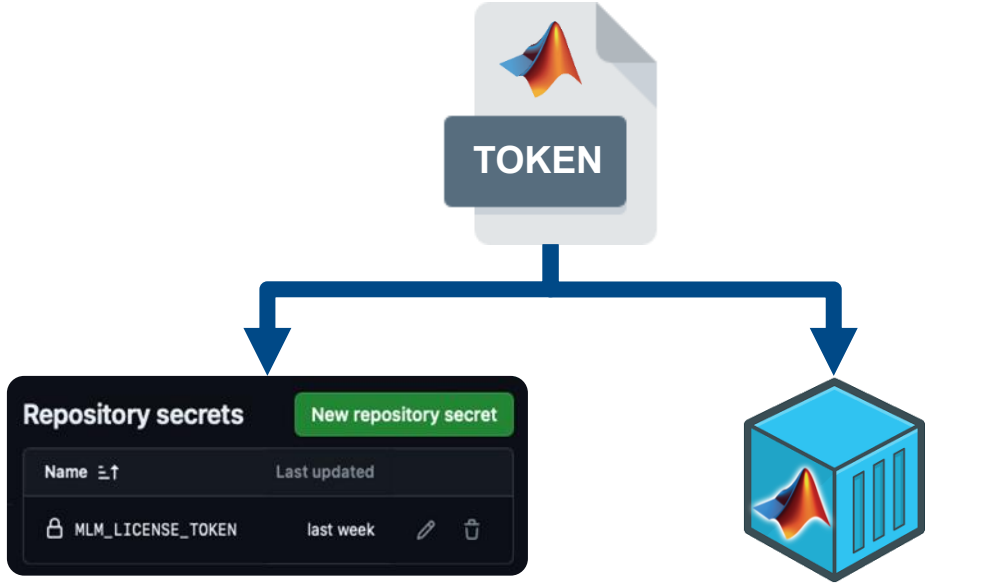
In Pilot

Batch License Token pilot available to enable modern scalability and cloud-native workflows

- New licensing mechanism built to better support CI/CD workflows

GitHub Actions circleci Azure Pipelines GitLab TC

- Batch Licensing Tokens…
  - Scale more efficiently than FlexLM
  - Can be used with hosted runners and cloud-native architectures



Repository secrets docker run

Batch Token Sign-Up Form
mathworks.com/support/batch-tokens

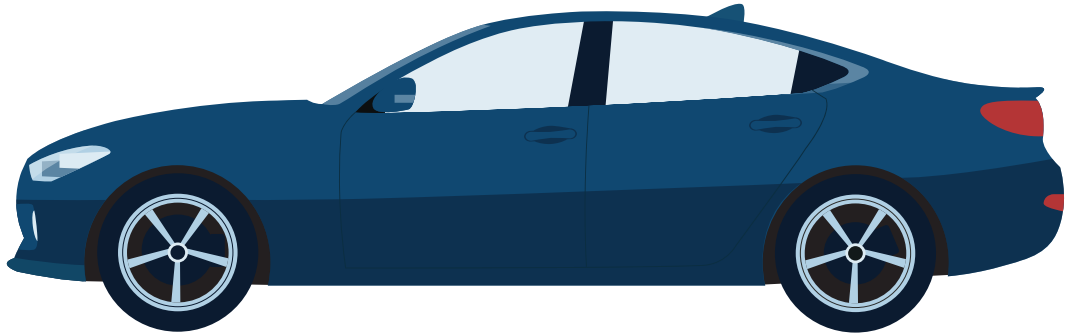
Executing MATLAB and Simulink in CI

- Available plugins for most major platforms
  - GitHub® Actions
  - Jenkins™
  - GitLab™
  - Azure® DevOps
  - CircleCI®
  - TeamCity®
- Example YAML files to help get you started
  - CI Configuration Examples
  - Advanced CI Configuration Examples
- Automatic YAML generation using CI Support Package for Simulink

CI in Action

Let's look at an example

- **"Launch Mode" for an Electric Vehicle**



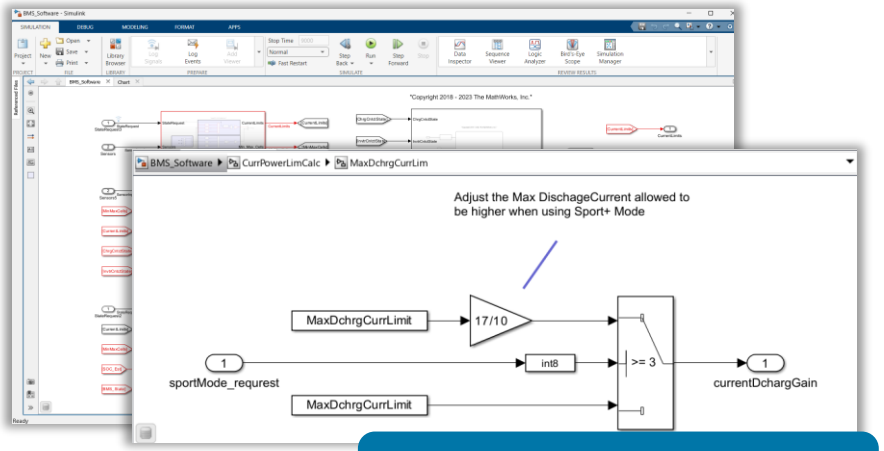
Driver Activates "Launch Mode"

↓

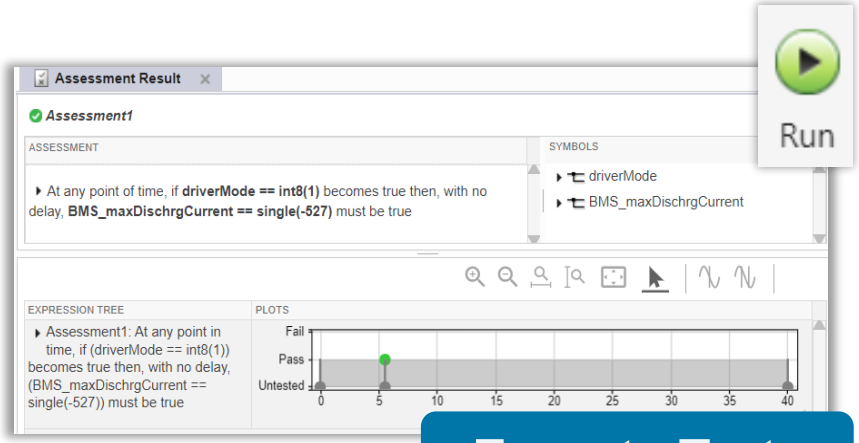
Increase Battery Current by 70%

A typical MBD workflow

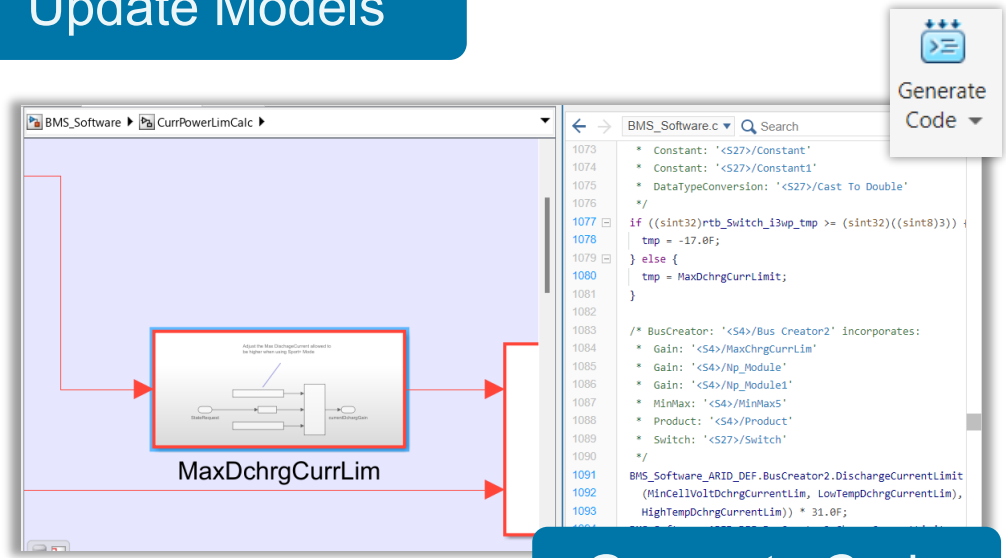
Implement requirements using Simulink Models, Tests and CodeGen



Update Models

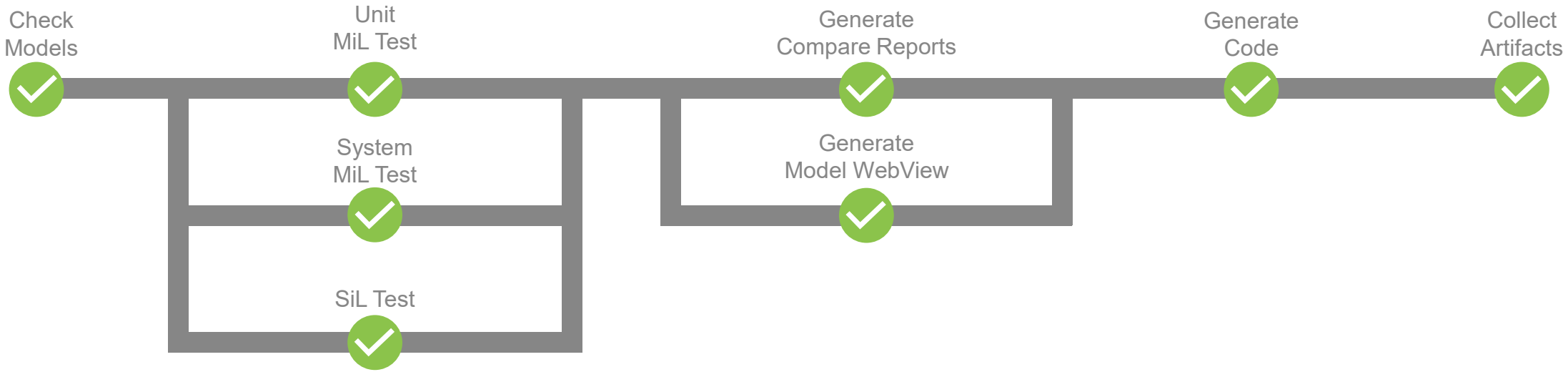


Execute Tests



Generate Code

Is my development work complete?

Identify tasks and define the process

- Which tasks need to be performed first?
- Which tasks are dependent?
- What can be executed simultaneously?